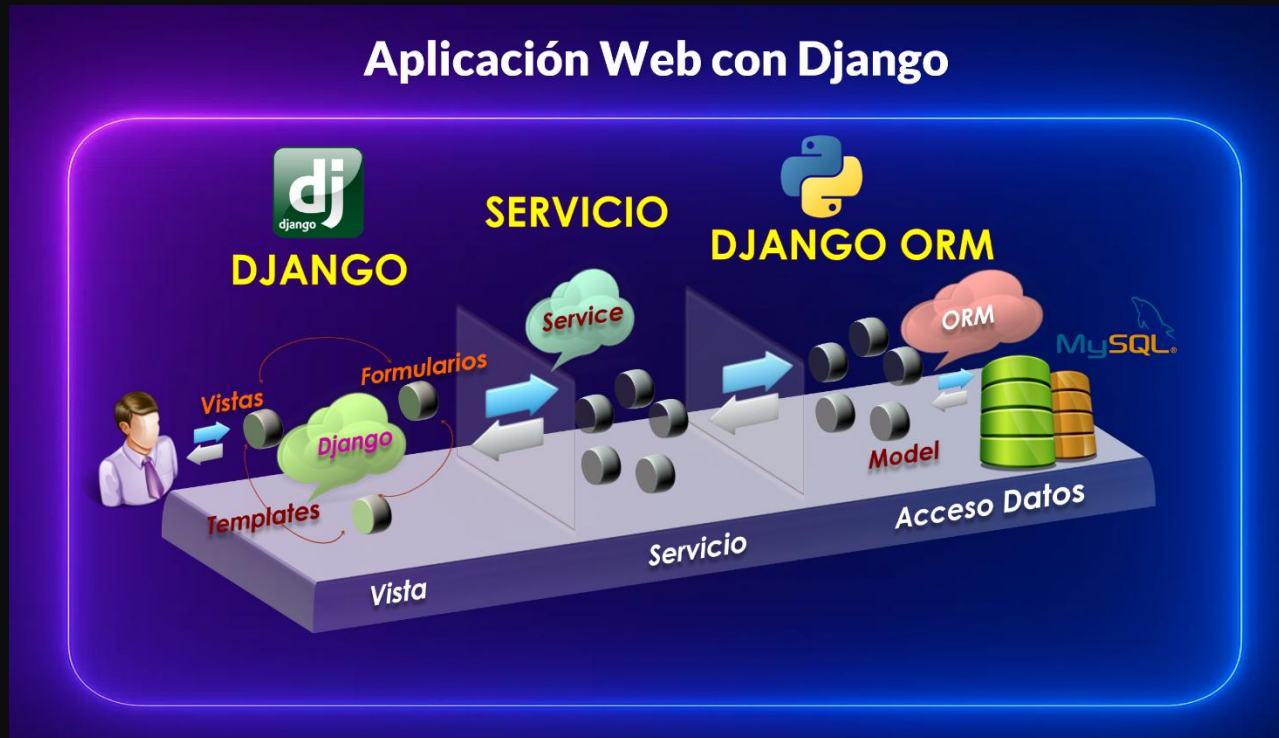




FASE 1: PREPARACIÓN E INSTALACIÓN

Objetivo de esta fase

Dejar funcionando un proyecto Django mínimo con una app llamada `canciones` y una portada simple que diga “Django Songs 🎧”, usando Bootstrap por CDN en **modo oscuro**.

Nota: Tal como pediste, en esta fase **no** creamos aún la carpeta de servicios ni la plantilla base (`base.html`). Mantendremos todo lo más simple posible.

1) Comandos (paso a paso)

1. Crear carpeta base del proyecto (si no existe):
2. `mkdir C:\Cursos\Django\DjangoSongs`
3. Instalar dependencias principales (Django, y dejamos listas PyMySQL y django-environ aunque se configuran en la Fase 2):
4. `pip install Django PyMySQL django-environ`
5. Crear el proyecto Django **dentro de** `C:\Cursos\Django\DjangoSongs` (el punto final `.` crea el proyecto en la carpeta actual en lugar de anidar otra subcarpeta):
6. `cd C:\Cursos\Django\DjangoSongs`
7. `django-admin startproject django_songs .`
8. Crear la aplicación principal `canciones`:
9. `python manage.py startapp canciones`
10. Crear la carpeta de plantillas (solo la necesaria para esta fase):

```
11. mkdir C:\Cursos\Django\DjangoSongs\templates
```

Con esto ya existe la estructura mínima. Ahora sí, **mostramos el código completo** de los archivos que se crean/modifican en esta fase.

2) Archivos (con rutas completas)

- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\django_songs\settings.py
- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\django_songs\urls.py
- **(NUEVO)** C:\Cursos\Django\DjangoSongs\canciones\urls.py
- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\views.py
- **(NUEVO)** C:\Cursos\Django\DjangoSongs\templates\index.html

No creamos aún: base.html, carpeta services, ni .env. Eso llegará en fases posteriores.

(MODIFICADO) C:\Cursos\Django\DjangoSongs\django_songs\settings.py

¿Qué cambiamos y por qué?

- Agregamos la app `canciones` en `INSTALLED_APPS` para que Django la reconozca.
- Indicamos a Django dónde buscar plantillas: la carpeta `templates` en la raíz del proyecto.
- (Opcional y útil) Ajustamos `LANGUAGE_CODE` a español y `TIME_ZONE` de CDMX.

Copia y pega completo:

```
from pathlib import Path
import os

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-reemplaza-esto-si-gustas' # Django genera una por
defecto al crear el proyecto.
DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # App del proyecto
    'canciones',
]
```

```

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'django_songs.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        # Carpeta global de plantillas
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'django_songs.wsgi.application'

# Base de datos: se configurará en FASE 2 (MySQL con PyMySQL). Por ahora, SQLite para
arrancar.
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3', # Temporal para probar arranque
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',},
]

# Ajustes regionales útiles (opcional)
LANGUAGE_CODE = 'es'
TIME_ZONE = 'America/Mexico_City'

USE_I18N = True
USE_TZ = True

STATIC_URL = 'static/'

```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\django_songs\urls.py

¿Qué hacemos y por qué?

Conectamos la ruta raíz (/) hacia las rutas de la app canciones. Así el index se atiende desde esa app.

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('canciones.urls')),
]
```

(NUEVO) C:\Cursos\Django\DjangoSongs\canciones\urls.py

¿Qué hacemos y por qué?

Definimos la URL raíz de la app que apunta a la vista index.

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\views.py

¿Qué hacemos y por qué?

Creamos la vista index que renderiza templates/index.html. Sin servicios aún, manteniéndolo simple.

```
from django.shortcuts import render

def index(request):
    # Portada simple; sin servicio ni base.html en esta fase
    return render(request, 'index.html')
```

(NUEVO) C:\Cursos\Django\DjangoSongs\templates\index.html

¿Qué hacemos y por qué?

Una portada mínima en **modo oscuro** con Bootstrap por **CDN**, centrada y con un título verde y el emoji 🎧. Aún no usamos base.html ni navbar (eso llegará más adelante).

```

<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Django Songs</title>
    <!-- Bootstrap CDN (modo oscuro activado por data-bs-theme="dark" en <html>) -->
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
      crossorigin="anonymous"
    >
    <style>
      .title-green { color: #20c997; } /* teal/verde agradable */
      .vh-center { min-height: 100vh; }
    </style>
  </head>
  <body class="bg-dark text-white d-flex align-items-center vh-center">
    <main class="container text-center">
      <h1 class="display-4 fw-bold title-green">Django Songs <span role="img" aria-
label="audífonos">🎧</span></h1>
      <p class="lead mt-3">Catálogo musical personal con Django.</p>
      <p class="text-secondary">Continuaremos con la conexión a MySQL en la siguiente
fase.</p>
    </main>

    <script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous">
    </script>
  </body>
</html>

```

3) Explicación breve de por qué

- **SECRET_KEY**: se usa para firmar cookies, sesiones y proteger formularios con CSRF. Django te generó una al crear el proyecto; más adelante (Fase 2) la moveremos a `.env` con `django-environ` para mayor seguridad.
 - **INSTALLED_APPS**: registramos `canciones` para que Django cargue sus modelos, señales y plantillas.
 - **TEMPLATES['DIRS']**: añadimos la carpeta global `templates` para centralizar vistas HTML.
 - **Sin `base.html` ni `navbar`**: cumpliendo tu regla, lo añadiremos recién en la **Fase 5 (Agregar)** para mantener la fase actual minimalista.
 - **Sin capa de servicios todavía**: la **Fase 4** es donde empezaremos con `services/song_service.py`.
-

4) Prueba rápida

1. Ejecuta el servidor:
2. `python manage.py runserver`
3. Visita: `http://127.0.0.1:8000/`
Debes ver la portada “Django Songs 🎧” en modo oscuro.

5) Errores típicos y solución

- **ModuleNotFoundError: No module named 'canciones'**
Asegúrate de haber corrido `python manage.py startapp canciones` y de haber agregado 'canciones' en `INSTALLED_APPS`.
- **No se encuentra la plantilla `index.html`**
Verifica que la carpeta sea exactamente `C:\Cursos\Django\DjangoSongs\templates\index.html` y que en `settings.py` esté `BASE_DIR / 'templates'` dentro de `TEMPLATES['DIRS']`.
- **El servidor inicia pero no carga Bootstrap**
Verifica que tengas acceso a Internet (Bootstrap viene por CDN) y que el atributo `data-bs-theme="dark"` esté en la etiqueta `<html>`.

¿Listo para continuar? Cuando confirmes con “listo”, avanzamos a la **FASE 2: Conexión con MySQL (PyMySQL + django-environ)**, donde:

- Crearemos el archivo `.env` (sin extensión).
- Configuraremos `settings.py` para leer variables de entorno.
- Activaremos el conector **PyMySQL** como reemplazo de `MySQLdb`.

Dijiste:

listo, avancemos a la fase 2

ChatGPT dijo:

FASE 2: CONEXIÓN CON MYSQL (usando PyMySQL)

Objetivo de esta fase

Configurar la conexión con MySQL utilizando PyMySQL y django-environ, registrando las credenciales en un archivo `.env` (sin extensión).

También probaremos que el servidor arranque correctamente leyendo las variables de entorno.

1) Comandos paso a paso

1. Si aún no lo hiciste, instala las librerías necesarias:
2. `pip install PyMySQL django-environ`
3. Crea la base de datos en MySQL Workbench o consola:
4. `CREATE DATABASE django_songs_db CHARACTER SET utf8mb4;`
5. Crea el archivo `.env` en la raíz del proyecto
(sin extensión):
6. `C:\Cursos\Django\DjangoSongs\.env`

Contenido:

```
DEBUG=True
DB_NAME=django_songs_db
DB_USER=root
DB_PASSWORD=admin
DB_HOST=localhost
DB_PORT=3306
```

7. Registra PyMySQL como reemplazo de MySQLdb (obligatorio en Windows):
Archivo a modificar:
`C:\Cursos\Django\DjangoSongs\django_songs\__init__.py`

2) Archivos modificados o creados

- **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\django_songs\__init__.py`
- **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\django_songs\settings.py`
- **(NUEVO/CONFIRMAR)** `C:\Cursos\Django\DjangoSongs\.env`

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\django_songs\__init__.py`

 Explicación:

Este archivo se ejecuta automáticamente al iniciar Django. Aquí registramos **PyMySQL** como si fuera el conector nativo `MySQLdb`, que Django espera por defecto.

Código completo:

```
import pymysql

# Registrar PyMySQL como reemplazo de MySQLdb
pymysql.install_as_MySQLdb()
```

(MODIFICADO) c:\Cursos\Django\DjangoSongs\django_songs\settings.py Explicación:

- Importamos `environ` para leer las variables desde `.env`.
- Configuramos `DATABASES` para usar **MySQL** con los datos del entorno.
- Eliminamos `SQLite` temporal (ya no se usará).

Código completo:

```

from pathlib import Path
import os
import environ

# --- Configuración base ---
BASE_DIR = Path(__file__).resolve().parent.parent

# --- Inicializar django-environ ---
env = environ.Env(
    DEBUG=(bool, False)
)
environ.Env.read_env(os.path.join(BASE_DIR, '.env'))

# --- Variables desde .env ---
SECRET_KEY = env('SECRET_KEY', default='django-insecure-dev-key')
DEBUG = env('DEBUG')

ALLOWED_HOSTS = []

# --- Aplicaciones ---
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'canciones',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'django_songs.urls'

```

```

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'django_songs.wsgi.application'

# --- Configuración de base de datos (MySQL con PyMySQL) ---
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST'),
        'PORT': env('DB_PORT'),
        'OPTIONS': {
            'charset': 'utf8mb4',
        },
    }
}

# --- Validaciones ---
AUTH_PASSWORD_VALIDATORS = [
    {'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

# --- Regionalización ---
LANGUAGE_CODE = 'es'
TIME_ZONE = 'America/Mexico_City'
USE_I18N = True
USE_TZ = True

STATIC_URL = 'static/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

```

(NUEVO/CONFIRMAR) C:\Cursos\Django\DjangoSongs\.env

⚠ Asegúrate de **no ponerle extensión** (ni `.txt` ni `.py`).
Este archivo guarda credenciales y configuraciones locales.

```
DEBUG=True
DB_NAME=django_songs_db
DB_USER=root
DB_PASSWORD=admin
DB_HOST=localhost
DB_PORT=3306
```

Explicación general

- **PyMySQL**: reemplaza a `MySQLdb`, que suele causar errores de compilación en Windows.
 - **django-environ**: permite leer configuraciones sensibles desde `.env` sin exponerlas en Git.
 - **.env sin extensión**: evita que Django lo interprete como un módulo o archivo ejecutable.
 - **DATABASES['default']**: define el motor, nombre de la BD y credenciales de conexión.
-

3) Prueba rápida

Ejecuta el servidor para comprobar la conexión:

```
python manage.py runserver
```

Abre en el navegador:

👉 <http://127.0.0.1:8000/>

Si todo está correcto, deberías ver la misma portada de la fase anterior, pero **ahora Django está leyendo la configuración desde MySQL** (aunque todavía no hay tablas).

4) Solución a errores comunes

- **Error: “django.core.exceptions.ImproperlyConfigured: Error loading MySQLdb module.”**
Asegúrate de tener en `__init__.py` las líneas:
 - `import pymysql`
 - `pymysql.install_as_MySQLdb()`
- **Error: “Access denied for user 'root'@'localhost'”**
Verifica las credenciales del `.env` y que MySQL esté en ejecución.
- **Error: “FileNotFoundError: .env not found”**
Confirma que `.env` está en `C:\Cursos\Django\DjangoSongs` y **no tiene extensión**.

¿Confirmamos que tu servidor arranca correctamente y puedes ver la portada?
Cuando me digas “**listo**”, pasamos a la **FASE 3: Creación del modelo `Cancion`** 🎵

Dijiste:

listo vamos a la fase 3

ChatGPT dijo:

🎵 FASE 3: CREACIÓN DEL MODELO

Objetivo de esta fase

Definir el modelo `Cancion` con validaciones de popularidad (1–10), aplicar migraciones para crear la tabla en MySQL y registrar el modelo en el panel de administración de Django.

📖 1) Comandos paso a paso

1. Abrir PyCharm y asegurarte de estar dentro del entorno virtual del proyecto.
Directorio base:
 2. `C:\Cursos\Django\DjangoSongs`
 3. Editar el archivo de modelo:
 4. `C:\Cursos\Django\DjangoSongs\canciones\models.py`
 5. Registrar el modelo en el panel de administración:
 6. `C:\Cursos\Django\DjangoSongs\canciones\admin.py`
 7. Crear migraciones:
 8. `python manage.py makemigrations`
 9. Aplicar migraciones (crear tabla en la base de datos):
 10. `python manage.py migrate`
 11. Crear un superusuario (opcional pero útil para pruebas):
 12. `python manage.py createsuperuser`
-

🧩 2) Archivos modificados o creados

- **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\models.py`
 - **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\admin.py`
-

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\canciones\models.py`

 Explicación

Este archivo define la estructura de la tabla `cancion` en la base de datos.

Incluimos validadores para asegurar que el campo `popularidad` esté entre 1 y 10, y configuramos un nombre de tabla personalizado para evitar nombres demasiado largos como `canciones_cancion`.

Código completo:

```
from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator

class Cancion(models.Model):
    titulo = models.CharField(max_length=100, verbose_name='Título')
    artista = models.CharField(max_length=100, verbose_name='Artista')
    popularidad = models.IntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(10)],
        verbose_name='Popularidad (1-10)'
    )

    class Meta:
        db_table = 'cancion' # Nombre de la tabla en MySQL
        verbose_name = 'Canción'
        verbose_name_plural = 'Canciones'

    def __str__(self):
        return f"{self.titulo} - {self.artista}"
```

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\canciones\admin.py`

 Explicación

Registramos el modelo `Cancion` para poder administrarlo desde el panel de Django.

Código completo:

```
from django.contrib import admin
from .models import Cancion

@admin.register(Cancion)
class CancionAdmin(admin.ModelAdmin):
    list_display = ('id', 'titulo', 'artista', 'popularidad')
    search_fields = ('titulo', 'artista')
```

3) Aplicar migraciones y verificar

1. Generar el archivo de migraciones:
2. `python manage.py makemigrations`

Si todo va bien, verás algo como:

```
Migrations for 'canciones':
  canciones/migrations/0001_initial.py
    - Create model Cancion
```

3. Aplicar migraciones:
4. `python manage.py migrate`

Django creará automáticamente la tabla `cancion` en tu base de datos MySQL.

5. Verifica la tabla en **MySQL Workbench**:
Deberías ver la estructura con los campos `id`, `titulo`, `artista`, `popularidad`.

4) Probar el panel de administración

1. Si no lo hiciste, crea un usuario administrador:
2. `python manage.py createsuperuser`

Completa con usuario, correo y contraseña.

3. Inicia el servidor:
4. `python manage.py runserver`
5. Entra al panel:
 `http://127.0.0.1:8000/admin`
6. Inicia sesión y deberías ver el módulo **Canciones** disponible.
Desde ahí podrás:
 - Agregar nuevas canciones.
 - Editarlas.
 - Eliminarlas.

5) Explicación adicional

- **Meta:** `db_table = 'cancion'` → asigna un nombre de tabla personalizado, evitando el formato largo predeterminado (`canciones_cancion`).
- **Validators** → garantizan que el valor de popularidad siempre esté dentro del rango de 1 a 10.
- **verbose_name / verbose_name_plural** → se usan para mostrar nombres legibles en el panel de administración.

6) Errores comunes

- **Error de conexión MySQL:** revisa tu `.env` y asegúrate de que MySQL esté activo.
 - **Tabla no creada:** asegúrate de haber ejecutado `makemigrations` seguido de `migrate`.
 - **Acceso al admin falla:** si olvidaste crear el superusuario, ejecútalo con `python manage.py createsuperuser`.
-

¿Listo para continuar?

Cuando confirmes con “**listo**”, pasamos a la **FASE 4: Caso de Listar (READ)** para mostrar las canciones en pantalla con Bootstrap 🎧

Dijiste:

listo vamos a la fase 4

ChatGPT dijo:



FASE 4: CASO DE LISTAR (READ)

Objetivo de esta fase

Mostrar en la página principal un **listado de canciones** obtenidas desde la base de datos, usando la capa de **servicio** para acceder al modelo.

Se implementará la primera interacción completa entre las tres capas del proyecto:

Modelo → Servicio → Vista → Plantilla.



1) Comandos paso a paso

1. Crear la carpeta `services` (si aún no existe):
 2. `mkdir C:\Cursos\Django\DjangoSongs\canciones\services`
 3. Crear el archivo `__init__.py` dentro de `services` (para que Python reconozca el módulo):
 4. `C:\Cursos\Django\DjangoSongs\canciones\services\__init__.py`
 5. Crear el archivo principal del servicio:
 6. `C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py`
 7. Modificar la vista y la plantilla `index.html` para mostrar los datos.
-



2) Archivos creados o modificados

- **(NUEVO)** `C:\Cursos\Django\DjangoSongs\canciones\services\__init__.py`
- **(NUEVO)** `C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py`
- **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\views.py`

- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\templates\index.html
-

(NUEVO) C:\Cursos\Django\DjangoSongs\canciones\services__init__.py

Archivo vacío, necesario para que Django reconozca la carpeta `services` como paquete Python:

```
# Este archivo permite que 'services' sea un módulo de Python
```

(NUEVO) C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py

✅ Explicación

Esta capa **separa la lógica de negocio** del acceso a la base de datos.

Aquí centralizamos las funciones CRUD. En esta fase solo implementamos `obtener_todas()`.

Código completo:

```
from canciones.models import Cancion

def obtener_todas():
    """Devuelve todas las canciones registradas ordenadas por ID."""
    return Cancion.objects.all().order_by('id')
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\views.py

✅ Explicación

La vista llama al servicio, obtiene los datos y los pasa a la plantilla a través del contexto.

Código completo:

```
from django.shortcuts import render
from canciones.services import song_service

def index(request):
    """Muestra el listado de canciones."""
    canciones = song_service.obtener_todas()
    contexto = {'canciones': canciones}
    return render(request, 'index.html', contexto)
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\templates\index.html

✅ Explicación

La plantilla recibe la lista `canciones` y las muestra en una tabla con Bootstrap en modo oscuro.

Si no hay canciones, muestra un mensaje amigable.

Código completo:

```

<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Listado de Canciones</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhJY6hW+ALEwIH"
      crossorigin="anonymous"
    >
  </head>
  <body class="bg-dark text-white">
    <div class="container py-5">
      <h1 class="text-center text-success mb-4">🎧 Django Songs</h1>
      <h5 class="text-center mb-4">Listado de canciones registradas</h5>

      {% if canciones %}
        <table class="table table-dark table-striped align-middle">
          <thead>
            <tr class="text-success">
              <th scope="col">ID</th>
              <th scope="col">Título</th>
              <th scope="col">Artista</th>
              <th scope="col">Popularidad</th>
            </tr>
          </thead>
          <tbody>
            {% for cancion in canciones %}
              <tr>
                <td>{{ cancion.id }}</td>
                <td>{{ cancion.titulo }}</td>
                <td>{{ cancion.artista }}</td>
                <td>{{ cancion.popularidad }}</td>
              </tr>
            {% endfor %}
          </tbody>
        </table>
      {% else %}
        <div class="alert alert-secondary text-center">
          No hay canciones registradas aún.
        </div>
      {% endif %}
    </div>

    <script
      src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
      integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
      crossorigin="anonymous">
    </script>

```

```
</body>
</html>
```

✓ 3) Verificación y prueba

1. Asegúrate de tener registros en la tabla `cancion` (puedes agregarlos desde el panel `/admin`).
2. Ejecuta el servidor:
3. `python manage.py runserver`
4. Abre en tu navegador:
👉 <http://127.0.0.1:8000/>

Deberías ver la tabla con todas tus canciones ordenadas por ID.
Si no hay registros, verás un mensaje de “No hay canciones registradas aún”.

💡 4) Conceptos clave

- **Capa de servicio:** evita mezclar lógica de negocio dentro de las vistas.
 - **Vistas simples (función `index`):** solo coordina la información entre servicio y plantilla.
 - **Plantilla `index`:** usa el lenguaje de plantillas de Django (`{% for ... %}`) para mostrar datos dinámicos.
 - **Separación clara de responsabilidades:**
 - Modelo: accede a la base de datos.
 - Servicio: contiene la lógica CRUD.
 - Vista: muestra datos y responde al usuario.
-

⚠ 5) Errores comunes

- **Error: "No module named 'canciones.services'"**
→ Revisa que exista el archivo vacío `__init__.py` dentro de `services`.
 - **Error de conexión MySQL**
→ Verifica que el servicio MySQL esté activo y que `.env` tenga las credenciales correctas.
 - **Página vacía sin canciones**
→ Asegúrate de haber agregado registros desde el panel `/admin`.
-

¿Listo para continuar?

Cuando confirmes con “**listo**”, avanzamos a la **FASE 5: Caso Agregar (CREATE)** 🚀 donde añadiremos un formulario con Django Forms para registrar nuevas canciones.

Dijiste:

listo vamos a la fase 5

ChatGPT dijo:



FASE 5: CASO AGREGAR (CREATE)

Objetivo de esta fase

Agregar nuevas canciones desde un **formulario con Django Forms**, usando la capa de **servicio** para guardar los datos y **redireccionar** al listado.

Aquí también añadiremos por primera vez la **plantilla base (base.html)** con Bootstrap y modo oscuro.



1) Comandos paso a paso

1. Verifica que estás en la carpeta del proyecto:
 2. `cd C:\Cursos\Django\DjangoSongs`
 3. No se requieren nuevos paquetes, ya que Django Forms viene integrado.
 4. Crea el formulario de Django:
 5. `C:\Cursos\Django\DjangoSongs\canciones\forms.py`
 6. Crea la plantilla del formulario:
 7. `C:\Cursos\Django\DjangoSongs\templates\canciones_form.html`
 8. Crea la plantilla base:
 9. `C:\Cursos\Django\DjangoSongs\templates\base.html`
 10. Modifica las rutas y vistas correspondientes.
-



2) Archivos modificados o creados

- **(NUEVO)** `C:\Cursos\Django\DjangoSongs\canciones\forms.py`
 - **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py`
 - **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\views.py`
 - **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\canciones\urls.py`
 - **(NUEVO)** `C:\Cursos\Django\DjangoSongs\templates\canciones_form.html`
 - **(NUEVO)** `C:\Cursos\Django\DjangoSongs\templates\base.html`
 - **(MODIFICADO)** `C:\Cursos\Django\DjangoSongs\templates\index.html`
-

(NUEVO) `C:\Cursos\Django\DjangoSongs\canciones\forms.py`

 Explicación

Definimos un `ModelForm` basado en el modelo `Cancion`.

Así Django generará automáticamente los campos del formulario y validará la entrada.

Código completo:

```
from django import forms
from canciones.models import Cancion

class CancionForm(forms.ModelForm):
    class Meta:
        model = Cancion
        fields = ['titulo', 'artista', 'popularidad']
        widgets = {
            'titulo': forms.TextInput(attrs={'class': 'form-control', 'placeholder':
'Título'}),
            'artista': forms.TextInput(attrs={'class': 'form-control', 'placeholder':
'Artista'}),
            'popularidad': forms.NumberInput(attrs={'class': 'form-control', 'min': 1,
'max': 10}),
        }
```

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py`

 Explicación

Agregamos la función `crear_cancion()` para registrar una nueva canción en la base de datos.

Código completo:

```
from canciones.models import Cancion

def obtener_todas():
    """Devuelve todas las canciones registradas ordenadas por ID."""
    return Cancion.objects.all().order_by('id')

def crear_cancion(datos):
    """Crea una nueva canción en la base de datos."""
    Cancion.objects.create(**datos)
```

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\canciones\views.py`

 Explicación

Creamos la vista `crear_cancion` que muestra el formulario y lo procesa.

- Si es `GET`, muestra el formulario vacío.
- Si es `POST`, guarda y redirige al listado.

Código completo:

```

from django.shortcuts import render, redirect
from canciones.services import song_service
from canciones.forms import CancionForm

def index(request):
    canciones = song_service.obtener_todas()
    contexto = {'canciones': canciones}
    return render(request, 'index.html', contexto)

def crear_cancion(request):
    """Vista para crear una nueva canción."""
    if request.method == 'POST':
        form = CancionForm(request.POST)
        if form.is_valid():
            song_service.crear_cancion(form.cleaned_data)
            return redirect('index')
    else:
        form = CancionForm()
        contexto = {'form': form, 'accion': 'Agregar Canción'}
        return render(request, 'canciones_form.html', contexto)

```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\urls.py

✓ Explicación

Agregamos una nueva ruta para crear canciones.

Código completo:

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('agregar/', views.crear_cancion, name='crear_cancion'),
]

```

(NUEVO) C:\Cursos\Django\DjangoSongs\templates\base.html

✓ Explicación

Plantilla base con navbar simple en modo oscuro.

Las demás plantillas extenderán de ella.

Código completo:

```

<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Django Songs{% endblock %}</title>

```

```

<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet"
  integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhJY6hW+ALEwIH"
  crossorigin="anonymous"
>
</head>
<body class="bg-dark text-white">
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark border-bottom mb-4">
    <div class="container">
      <a class="navbar-brand text-success fw-bold" href="{% url 'index' %}">🎧 Django
Songs</a>
      <div class="d-flex">
        <a href="{% url 'crear_cancion' %}" class="btn btn-success">+ Agregar
Canción</a>
      </div>
    </div>
  </nav>

  <main class="container">
    {% block content %}{% endblock %}
  </main>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous">
  </script>
</body>
</html>

```

(NUEVO) C:\Cursos\Django\DjangoSongs\templates\canciones_form.html

✅ Explicación

Plantilla para agregar (y reutilizable para editar) canciones.

Usamos `{{ form.as_p }}` para generar los campos con etiquetas `<p>` y aplicamos Bootstrap.

Código completo:

```

{% extends 'base.html' %}

{% block title %}{{ accion }}{% endblock %}

{% block content %}
  <h2 class="text-center text-success mb-4">{{ accion }}</h2>
  <form method="post" class="mx-auto" style="max-width: 500px;">
    {% csrf_token %}
    {{ form.as_p }}
    <div class="text-center mt-3">
      <button type="submit" class="btn btn-success">Guardar</button>
    </div>
  </form>
{% endblock %}

```

```

    <a href="{% url 'index' %}" class="btn btn-secondary">Cancelar</a>
  </div>
</form>
{% endblock %}

```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\templates\index.html

✓ Explicación

Ahora extiende de `base.html` para reutilizar la estructura y el botón de agregar.

Código completo:

```

{% extends 'base.html' %}

{% block title %}Listado de Canciones{% endblock %}

{% block content %}
  <h3 class="text-center mb-4">Listado de canciones registradas</h3>

  {% if canciones %}
    <table class="table table-dark table-striped align-middle">
      <thead>
        <tr class="text-success">
          <th>ID</th>
          <th>Título</th>
          <th>Artista</th>
          <th>Popularidad</th>
        </tr>
      </thead>
      <tbody>
        {% for cancion in canciones %}
          <tr>
            <td>{{ cancion.id }}</td>
            <td>{{ cancion.titulo }}</td>
            <td>{{ cancion.artista }}</td>
            <td>{{ cancion.popularidad }}</td>
          </tr>
        {% endfor %}
      </tbody>
    </table>
  {% else %}
    <div class="alert alert-secondary text-center">
      No hay canciones registradas aún.
    </div>
  {% endif %}
{% endblock %}

```

✓ 3) Prueba rápida

1. Ejecuta el servidor:

2. `python manage.py runserver`
3. En el navegador, entra a:
 <http://127.0.0.1:8000/>
4. Haz clic en “+ Agregar Canción”, completa el formulario y guarda.
5. Deberías ser redirigido al listado con la nueva canción visible.

4) Conceptos importantes

- **Formularios de Django** validan automáticamente los datos del modelo.
- **form.as_p** → genera el formulario con cada campo dentro de un `<p>`.
- **Capa de servicio** mantiene el control del acceso al modelo, separando lógica del controlador.
- **Plantilla base** permite reutilizar la estructura visual en todas las páginas.

5) Errores comunes

- “**No module named canciones.forms**” → verifica la ruta y nombre exacto del archivo `forms.py`.
- **Formulario vacío después de guardar** → asegúrate de redirigir con `return redirect('index')`.
- **Error CSRF** → verifica que el formulario incluya `{% csrf_token %}`.

¿Listo para continuar?

Cuando confirmes con “**listo**”, avanzamos a la **FASE 6: Caso Editar (UPDATE)**  donde aprenderás a reutilizar el mismo formulario para actualizar canciones existentes.

Dijiste:

listo vamos a la fase 6

ChatGPT dijo:

FASE 6: CASO EDITAR (UPDATE)

Objetivo de esta fase

Permitir **editar canciones existentes** reutilizando el mismo formulario creado en la fase anterior.

La vista precargará los datos de la canción seleccionada, y al guardar se actualizará en la base de datos mediante la **capa de servicio**.

1) Comandos paso a paso

1. Verifica que el proyecto sigue en ejecución correctamente.
Directorio base:
 2. C:\Cursos\Django\DjangoSongs
 3. No se necesitan nuevos comandos, solo modificaremos archivos existentes.
-

2) Archivos modificados

- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py
 - **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\views.py
 - **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\urls.py
 - **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\templates\canciones_form.html
 - **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\templates\index.html
-

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py

Explicación

Agregamos la función `actualizar_cancion()` que busca la canción por su ID, asigna los nuevos valores y guarda los cambios.

Código completo:

```
from canciones.models import Cancion

def obtener_todas():
    """Devuelve todas las canciones registradas ordenadas por ID."""
    return Cancion.objects.all().order_by('id')

def crear_cancion(datos):
    """Crea una nueva canción en la base de datos."""
    Cancion.objects.create(**datos)

def actualizar_cancion(id, datos):
    """Actualiza los datos de una canción existente."""
    cancion = Cancion.objects.get(pk=id)
    for campo, valor in datos.items():
        setattr(cancion, campo, valor)
    cancion.save()
    return cancion
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\views.py

 Explicación

Se agrega la vista `editar_cancion()`, que:

- Obtiene la canción por su ID.
- Si es `GET`, muestra el formulario precargado.
- Si es `POST`, guarda los cambios usando el servicio y redirige al listado.

Código completo:

```
from django.shortcuts import render, redirect, get_object_or_404
from canciones.services import song_service
from canciones.forms import CancionForm

def index(request):
    canciones = song_service.obtener_todas()
    contexto = {'canciones': canciones}
    return render(request, 'index.html', contexto)

def crear_cancion(request):
    if request.method == 'POST':
        form = CancionForm(request.POST)
        if form.is_valid():
            song_service.crear_cancion(form.cleaned_data)
            return redirect('index')
    else:
        form = CancionForm()
        contexto = {'form': form, 'accion': 'Agregar Canción'}
        return render(request, 'canciones_form.html', contexto)

def editar_cancion(request, id):
    """Vista para editar una canción existente."""
    cancion = get_object_or_404(song_service.obtener_todas(), pk=id)
    if request.method == 'POST':
        form = CancionForm(request.POST, instance=cancion)
        if form.is_valid():
            song_service.actualizar_cancion(id, form.cleaned_data)
            return redirect('index')
    else:
        form = CancionForm(instance=cancion)
        contexto = {'form': form, 'accion': 'Editar Canción'}
        return render(request, 'canciones_form.html', contexto)
```

(MODIFICADO) `C:\Cursos\Django\DjangoSongs\canciones\urls.py` Explicación

Se añade la ruta para editar canciones, recibiendo el parámetro `<int:id>`.

Código completo:

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('', views.index, name='index'),
    path('agregar/', views.crear_cancion, name='crear_cancion'),
    path('editar/<int:id>/', views.editar_cancion, name='editar_cancion'),
]
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\templates\canciones_form.html

✓ Explicación

Solo cambiamos el texto del botón para que se adapte automáticamente según la acción.

Código completo:

```
{% extends 'base.html' %}

{% block title %}{{ accion }}{% endblock %}

{% block content %}
    <h2 class="text-center text-success mb-4">{{ accion }}</h2>
    <form method="post" class="mx-auto" style="max-width: 500px;">
        {% csrf_token %}
        {{ form.as_p }}
        <div class="text-center mt-3">
            <button type="submit" class="btn btn-success">
                {% if accion == 'Editar Canción' %}Actualizar{% else %}Guardar{% endif %}
            </button>
            <a href="{% url 'index' %}" class="btn btn-secondary">Cancelar</a>
        </div>
    </form>
{% endblock %}
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\templates\index.html

✓ Explicación

Agregamos un botón “Editar” junto a cada registro.

Cada botón redirige a /editar/<id>/.

Código completo:

```
{% extends 'base.html' %}

{% block title %}Listado de Canciones{% endblock %}

{% block content %}
    <h3 class="text-center mb-4">Listado de canciones registradas</h3>

    {% if canciones %}
        <table class="table table-dark table-striped align-middle">
            <thead>
```

```

<tr class="text-success">
    <th>ID</th>
    <th>Título</th>
    <th>Artista</th>
    <th>Popularidad</th>
    <th>Acciones</th>
</tr>
</thead>
<tbody>
    {% for cancion in canciones %}
        <tr>
            <td>{{ cancion.id }}</td>
            <td>{{ cancion.titulo }}</td>
            <td>{{ cancion.artista }}</td>
            <td>{{ cancion.popularidad }}</td>
            <td>
                <a href="{% url 'editar_cancion' cancion.id %}" class="btn btn-warning
btn-sm">Editar</a>
            </td>
        </tr>
    {% endfor %}
</tbody>
</table>
{% else %}
    <div class="alert alert-secondary text-center">
        No hay canciones registradas aún.
    </div>
{% endif %}
{% endblock %}

```

✓ 3) Prueba rápida

1. Inicia el servidor:
2. `python manage.py runserver`
3. Visita:
 <http://127.0.0.1:8000/>
4. Haz clic en **Editar** junto a una canción.
El formulario se mostrará con los datos precargados.
5. Modifica los valores y presiona **Actualizar**.
Serás redirigido al listado con los cambios reflejados.

💡 4) Conceptos importantes

- **Reutilización del formulario:** con el mismo `CancionForm`, manejamos tanto la creación como la edición.
- **instance=cancion:** Django rellena el formulario con los datos actuales del objeto.
- **get_object_or_404():** devuelve el objeto o muestra un error 404 si no existe.

- **Redirección:** después de actualizar, volvemos al listado para mantener la navegación fluida.

5) Errores comunes

- **Error “CancionForm() takes no keyword argument 'instance'”**
→ Asegúrate de que `CancionForm` herede de `forms.ModelForm`.
- **Al editar, no cambia la información**
→ Verifica que la función `actualizar_cancion` use `save()` correctamente.
- **Botón de editar no funciona**
→ Confirma que la URL usa `<int:id>` y que en el `href` se pasa `cancion.id`.

¿Listo para continuar?

Cuando confirmes con “**listo**”, avanzamos a la **FASE 7: Caso Eliminar (DELETE)**  para completar el CRUD y agregar confirmación visual antes de borrar canciones.

Dijiste:

listo vamos a la fase 7

ChatGPT dijo:



FASE 7: CASO ELIMINAR (DELETE)

Objetivo de esta fase

Implementar la funcionalidad para **eliminar canciones** desde el listado, usando un **botón con confirmación** y una petición **POST protegida por CSRF**.

Con esto completaremos el CRUD completo de Django Songs .

1) Comandos paso a paso

1. No es necesario crear nuevos archivos.
Solo modificaremos los existentes para añadir la función de eliminar.

2) Archivos modificados

- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py
- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\views.py
- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\canciones\urls.py
- **(MODIFICADO)** C:\Cursos\Django\DjangoSongs\templates\index.html

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\services\song_service.py

✅ Explicación

Agregamos la función `eliminar_cancion()` que recibe el ID, busca la canción y la elimina.

Código completo:

```
from canciones.models import Cancion

def obtener_todas():
    """Devuelve todas las canciones registradas ordenadas por ID."""
    return Cancion.objects.all().order_by('id')

def crear_cancion(datos):
    """Crea una nueva canción en la base de datos."""
    Cancion.objects.create(**datos)

def actualizar_cancion(id, datos):
    """Actualiza los datos de una canción existente."""
    cancion = Cancion.objects.get(pk=id)
    for campo, valor in datos.items():
        setattr(cancion, campo, valor)
    cancion.save()
    return cancion

def eliminar_cancion(id):
    """Elimina una canción existente por su ID."""
    cancion = Cancion.objects.get(pk=id)
    cancion.delete()
```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\views.py

✅ Explicación

Se agrega la vista `eliminar_cancion()`, que maneja la solicitud **POST** para borrar una canción. Luego redirige al listado.

Código completo:

```
from django.shortcuts import render, redirect, get_object_or_404
from canciones.services import song_service
from canciones.forms import CancionForm

def index(request):
```

```

canciones = song_service.obtener_todas()
contexto = {'canciones': canciones}
return render(request, 'index.html', contexto)

def crear_cancion(request):
    if request.method == 'POST':
        form = CancionForm(request.POST)
        if form.is_valid():
            song_service.crear_cancion(form.cleaned_data)
            return redirect('index')
    else:
        form = CancionForm()
        contexto = {'form': form, 'accion': 'Agregar Canción'}
        return render(request, 'canciones_form.html', contexto)

def editar_cancion(request, id):
    cancion = get_object_or_404(song_service.obtener_todas(), pk=id)
    if request.method == 'POST':
        form = CancionForm(request.POST, instance=cancion)
        if form.is_valid():
            song_service.actualizar_cancion(id, form.cleaned_data)
            return redirect('index')
    else:
        form = CancionForm(instance=cancion)
        contexto = {'form': form, 'accion': 'Editar Canción'}
        return render(request, 'canciones_form.html', contexto)

def eliminar_cancion(request, id):
    """Elimina una canción por método POST."""
    if request.method == 'POST':
        song_service.eliminar_cancion(id)
        return redirect('index')

```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\canciones\urls.py

✅ Explicación

Agregamos la ruta para eliminar canciones mediante POST.

Código completo:

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('agregar/', views.crear_cancion, name='crear_cancion'),
    path('editar/<int:id>/', views.editar_cancion, name='editar_cancion'),
    path('eliminar/<int:id>/', views.eliminar_cancion, name='eliminar_cancion'),
]

```

(MODIFICADO) C:\Cursos\Django\DjangoSongs\templates\index.html

✅ Explicación

Agregamos un botón **Eliminar** que abre un modal de confirmación.
El modal contiene un formulario con **POST** y token **CSRF**.

Código completo:

```
{% extends 'base.html' %}

{% block title %}Listado de Canciones{% endblock %}

{% block content %}
    <h3 class="text-center mb-4">Listado de canciones registradas</h3>

    {% if canciones %}
        <table class="table table-dark table-striped align-middle">
            <thead>
                <tr class="text-success">
                    <th>ID</th>
                    <th>Título</th>
                    <th>Artista</th>
                    <th>Popularidad</th>
                    <th>Acciones</th>
                </tr>
            </thead>
            <tbody>
                {% for cancion in canciones %}
                    <tr>
                        <td>{{ cancion.id }}</td>
                        <td>{{ cancion.titulo }}</td>
                        <td>{{ cancion.artista }}</td>
                        <td>{{ cancion.popularidad }}</td>
                        <td>
                            <a href="{% url 'editar_cancion' cancion.id %}" class="btn btn-warning btn-sm">Editar</a>

                            <!-- Botón que abre el modal -->
                            <button type="button" class="btn btn-danger btn-sm" data-bs-
toggle="modal" data-bs-target="#modalEliminar{{ cancion.id }}">
                                Eliminar
                            </button>

                            <!-- Modal de confirmación -->
                            <div class="modal fade" id="modalEliminar{{ cancion.id }}" tabindex="-1"
aria-hidden="true">
                                <div class="modal-dialog modal-dialog-centered">
                                    <div class="modal-content bg-dark text-white">
                                        <div class="modal-header border-secondary">
                                            <h5 class="modal-title text-danger">Confirmar eliminación</h5>
                                            <button type="button" class="btn-close btn-close-white" data-bs-
dismiss="modal"></button>
                                        </div>
                                        <div class="modal-body">
                                            ¿Estás seguro de que deseas eliminar la canción
                                            <strong class="text-success">{{ cancion.titulo }}</strong>?
                                        </div>
                                    </div>
                                </div>
                            </div>
                        </td>
                    </tr>
                {% endfor %}
            </tbody>
        </table>
    {% else %}
        <p>No hay canciones registradas.</p>
    {% endif %}
{% endblock %}
```

```

        <div class="modal-footer border-secondary">
            <form method="post" action="{% url 'eliminar_cancion' cancion.id
{%}">

                {% csrf_token %}
                <button type="submit" class="btn btn-danger">Eliminar</button>
            </form>
            <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Cancelar</button>
        </div>
    </div>
</div>
</div>
</td>
</tr>
{% endfor %}
</tbody>
</table>
{% else %}
    <div class="alert alert-secondary text-center">
        No hay canciones registradas aún.
    </div>
{% endif %}
{% endblock %}

```

✓ 3) Prueba rápida

1. Ejecuta el servidor:
 2. `python manage.py runserver`
 3. Visita el listado:
 <http://127.0.0.1:8000/>
 4. Haz clic en el botón **Eliminar** de una canción.
Aparecerá un **modal de confirmación**.
 5. Si confirmas, la canción será eliminada y volverás al listado sin ese registro.
-

💡 4) Conceptos importantes

- **Método POST:** es la forma segura de eliminar datos (no se usa GET para borrar).
 - **CSRF token:** protege contra ataques de falsificación de formularios.
 - **Modal Bootstrap:** mejora la experiencia del usuario mostrando una confirmación visual.
 - **Capa de servicio:** mantiene la lógica de eliminación separada de la vista.
-

⚠ 5) Errores comunes

- “Method Not Allowed (GET)”
→ Asegúrate de que el formulario de eliminación tenga `method="post"`.
 - El modal no aparece
→ Verifica que Bootstrap JS esté cargado correctamente en `base.html`.
 - La canción no se elimina
→ Confirma que el ID sea correcto y que la función `eliminar_cancion` llame a `.delete()`.
-

¡FELICIDADES!

Has completado con éxito las **7 fases del CRUD Django Songs**.
Ahora tienes una aplicación totalmente funcional con:

- Modelo validado 
- Conexión MySQL segura 
- Capa de servicios separada 
- Formularios Django con Bootstrap 
- Operaciones CRUD completas (Create, Read, Update, Delete) 

 **Django Songs** es una base excelente para proyectos más grandes.

Explicaciones complementarias:

En Django, es **fundamental entender la diferencia entre un *proyecto* y una *aplicación***, porque toda la arquitectura se basa en esa separación. Vamos a desglosarlo paso a paso, tomando como ejemplo la estructura que aparece en tu captura (`DjangoSongs`).

1. Proyecto Django

Un **proyecto** es el **contenedor general** que agrupa toda tu aplicación web.
Contiene la **configuración global**: base de datos, rutas principales, plantillas, seguridad, etc.

En tu caso, el proyecto se llama:

`DjangoSongs/`

Dentro de él está el directorio principal del mismo nombre (`django_songs`), que **se genera automáticamente** con el comando:

```
django-admin startproject django_songs
```

✿ Archivos principales del proyecto (django_songs/)

Archivo	Función
<code>__init__.py</code>	Indica que es un paquete de Python.
<code>settings.py</code>	✿ Contiene la configuración global del proyecto: base de datos, apps instaladas, rutas a plantillas, idioma, zona horaria, etc.
<code>urls.py</code>	🌐 Enruta las peticiones principales del proyecto hacia las aplicaciones registradas. Es el “mapa” de URLs.
<code>asgi.py</code>	🚀 Permite servir la aplicación de manera asíncrona (útil para WebSockets o servidores ASGI).
<code>wsgi.py</code>	🧱 Sirve la aplicación en producción usando servidores compatibles con WSGI (Gunicorn, mod_wsgi, etc.).

Y fuera del paquete (a nivel raíz del proyecto):

Archivo	Función
<code>manage.py</code>	🚩 Es el punto de entrada para comandos administrativos (<code>runserver</code> , <code>migrate</code> , <code>createsuperuser</code> , etc.).

🧱 2. Aplicación Django

Una **aplicación** es un **módulo reutilizable** dentro del proyecto, con una función específica (por ejemplo: usuarios, productos, canciones, etc.).

En tu estructura tienes la app:

```
canciones/
```

La creaste con el comando:

```
python manage.py startapp canciones
```

Cada aplicación puede funcionar de forma independiente dentro del proyecto, siempre que esté registrada en `settings.py` dentro de `INSTALLED_APPS`.

✿ Archivos principales de una aplicación (canciones/)

Archivo	Función
<code>models.py</code>	 Define las clases que representan tablas en la base de datos (el <i>modelo de datos</i>).
<code>views.py</code>	 Contiene las funciones o clases que responden a las solicitudes del usuario (controladores).
<code>urls.py</code> (si lo creas)	 Define las rutas específicas de la aplicación. Luego se conecta con el <code>urls.py</code> principal.
<code>forms.py</code> (opcional)	 Permite crear formularios de Django (con validaciones y widgets).
<code>admin.py</code>	 Registra tus modelos en el panel de administración de Django.
<code>apps.py</code>	 Configura metadatos de la app, como el nombre y etiqueta.
<code>migrations/</code>	 Contiene los archivos de migración que Django usa para sincronizar los modelos con la base de datos.
<code>templates/</code> (puede estar aquí o a nivel de proyecto)	 Contiene las plantillas HTML que renderizan las vistas.

3. Relación entre proyecto y aplicación

Podemos verlo como **una jerarquía funcional**:

```

Proyecto DjangoSongs
├── Configura entorno (settings.py)
├── Controla las rutas globales (urls.py)
└── Incluye aplicaciones
    ├── canciones/
    │   ├── models.py → Base de datos
    │   ├── views.py → Lógica y controladores
    │   └── templates/ → Interfaz HTML

```

En resumen:

- El **proyecto** es el “cerebro y sistema operativo”.
- Cada **aplicación** es un “módulo funcional” que realiza una tarea específica.

Ejemplo de flujo de trabajo

1. El usuario accede a `http://localhost:8000/canciones/`.
2. `django_songs/urls.py` redirige esa ruta al `urls.py` de la app `canciones`.

3. `canciones/views.py` procesa la solicitud (consulta el modelo, prepara datos).
4. `models.py` obtiene los registros desde la base de datos.
5. La vista renderiza una plantilla en `templates/`.
6. Django devuelve la respuesta al navegador.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)